



Instructor: Date:

Learner: Regional Branch:

Program: Center:

CONTENT

- Let's Start! • Let's Explore! • Let's Practice! • Extension Activity • Target Words • Learning Points



Let's Start!

UML Diagrams and Their Syntax



Learning Outcome:

Describe common simple incidents at work in a simple spoken or written situation with a supportive interlocutor. Build the macro system conceptual model following the customer requirements, through the use and interpretation of collected information, represented in class diagrams, interaction diagrams, and operation contracts.

Can Do Statements:

- I can say how to draw a class diagram using UML syntax.
- I can talk about how to denote relationships between classes.
- I can describe characteristics for an association relationship in a diagram.

Ask your partner:

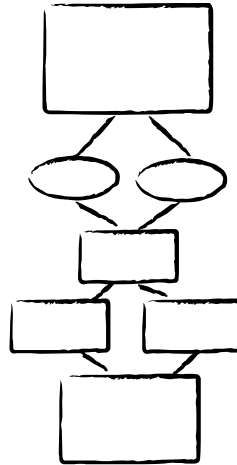
- Why do you think it is important to diagram the functional structure of a program?
- What do you use diagrams for in object oriented programming?





Let's Explore!

Jeremy explains to his students how UML classes work.



Instructions: Read the text below.

What is UML? Well, UML stands for Unified Modeling Language, that is, a programming language that you can use to model programs, designing the way they work and communicate with other platforms based on different languages, such as C#, Java, or C++, among others.

While you diagram your software with UML, you need to know some basic concepts that can help you during the process. First, you need to know that UML, as a language that it is, has syntax or shapes that allow you to understand how it works. For example, UML Classes. A class is an object that contains three main concepts: class name, attribute, and method. Each class represents a process within a system. Let's say, for example, a program that organizes the users, products, and purchases on a bookstore. Here, the program needs a user class; that class needs some attributes, as for example, name, age, and gender; and some methods, like, buy, retrieve, or purchase. In terms of UML syntax, the attributes need to have specific parameters, like for example, visibility or multiplicity. If I want the program to share some attributes with other classes, I can use the "public" parameter, or if I only want the class to have access to that attribute, I use the "private" one.

Obviously, the user class doesn't work isolated, it can have some relationship with other classes; for example, a *Book* class. Now, to create a direct relationship or association between both classes, the book class needs to have a name character string or age int variable. That way, they can relate one another; similarly, you can denote the relationship between the classes by adding different kinds of pointers as well as alphanumeric symbols to show the multiplicity relationship, as there can be dependency, simple, bidirectional, and composition association types, all denoted by the pointers. Such association types show how classes share their attributes one another.



Let's Practice!

Let's talk about how to denote relationships between classes!

Instructions: Read the paragraph. Underline all the mistakes in the paragraph.

1. You can detune some parameters as public by adding a + symbol before them.
2. If you want only the class to have access to a specific parameter, you cam use the "private" attribute.
3. You can deceive what attribute to give to each of the parameters or methods of the class.



Let's Practice!

Let's remember what Jeremy says about classes association.

Instructions: Read the sentences. Write the missing word(s) in the blank. Use: link classes, classes have, describe the multiplicity. Practice saying the sentences aloud. Check your pronunciation with your instructor.

The association that two or more _____ is the same relation you create by adding pointers that _____ and subclasses together. Those pointers or arrows usually _____ relation they can have, that is, the relation in terms of alphanumerical quantities.



Extension Activity

It's time to see how much you know about UML classes.

Instructions: Complete the following steps. Fill in the Self-Assessment.

- Search more about characteristics of diagrams association in UML.
- Write a short paragraph about two of the main characteristics of diagrams association in UML.
- Use the grammar and the vocabulary of this lesson.
- Share the short paragraph with your teacher.

Self-Assessment

Yes

No

- I do research about characteristics of diagrams association in UML.
- I write a short paragraph about diagrams association in UML.
- I use the grammar and the vocabulary of this lesson.
- I share the short paragraph with my teacher.

☐ ☐
☐ ☐
☐ ☐
☐ ☐


Target Words

- syntax** (noun): the shape or structure of a programming language
e.g. In terms of UML syntax, the attributes need to have specific parameters, like for example, visibility or multiplicity.
- visibility** (noun): access properties of an attribute
e.g. In terms of UML syntax, the attributes need to have specific parameters, like for example, visibility or multiplicity.
- alphanumeric** (adjective): something that combines alphabetic and numeric characters
e.g. You can denote the relationship between the classes by adding different kinds of pointers as well as alphanumeric symbols of the multiplicity.
- multiplicity** (noun): variety in number
e.g. You can denote the relationship between the classes by adding different kinds of pointers as well as alphanumeric symbols of the multiplicity.
- int** (noun): integer number
e.g. The book class needs to have a name character string or age int variable.
- character string** (noun): symbols in a specific sequence
e.g. The book class needs to have a name character string or age int variable.



Learning Points

Can for Possibilities

When you want to express the **possibility** someone has over something, you can use the modal auxiliary verb **can**.

Use the modal verb **can** to express the possibilities someone or something has over a situation.

In affirmative statements use the following structure:

Subject + Modal Verb Can + Verb in base form + Complement

Remember that no matter what subject you use. This model verb doesn't change.

Examples:

*You **can relate** any class that you think has a direct relationship with this object.*

*It **can work** if you link the correct classes.*

Examples

- *You need to know some basic concepts that can help you during the process.*
- *If I want the program to share some attributes with other classes, I can use the "public" parameter.*
- *You can denote the relationship between the classes by adding different kinds of pointers as well as alphanumeric symbols.*



Learning Points

Using the Correct UML Syntax in a Diagram

As any language, UML has its own syntax, that is, the way you represent the diagrams so that people can understand how it works.

It is important to know how to use **the correct UML syntax for a diagram** because that way, you can have a standard to organize and explain how the program works within a system.

The most common syntax you can find in UML is the **Class diagrams**. You can identify a class diagram by its shape, which is commonly a rectangular chart where you find the following information: Class name, Attributes, and Methods.

The first step to follow is to organize the class by having clear the **Class name**; here you can include the name of the object that the program needs to carry out; for example, User.

Next, you need to understand that the **Attributes** are the processes or main characteristics the class has; for example, name, age, gender, or status.

Similarly, the **Methods** are the properties the class has regarding the attributes; for example, Request, Enroll or Expel. That, depending on the purpose of the program.

On the other hand, you need to have clear that attributes and methods use some specific keywords within the syntax of the UML, for example, the command 'int' that represents an integer number or the string command, among many others.

In the following chart, you can see how the syntax of a class diagram works, on the left you can find a main class with: Classname, attributes, and methods.

On the right, there is a subclass that shares or requests some attributes from the first one.

The symbols before the attributes and methods are the parameters that indicate if that information is visible or private to other subclasses. Similarly, the arrow linking the classes represents the type of relation they share as well as the multiplicity that the numbers represent.



Learning Points

